Tamp v0.8.0: Measured Token Savings for Agentic Coding Assistants

Stas Kulesh — x.com/staskulesh

April 21, 2026

Abstract. *Tamp v0.8.0 ships a seventeen-stage HTTP-proxy compression pipeline exposed as a nine-level ladder (L1–L9). In a controlled live benchmark (12 scenarios × 18 configs = 216 A/B calls routed through OpenRouter, judged by Claude Sonnet Haiku 4.5), every configuration preserves task-completion quality on 216/216 tasks. At the balanced default (L5) we measure 45.34% bytes saved / 47.56% tokens saved; the ceiling at L9 is 45.39% / 47.61%, only +0.24 percentage points above the v0.5 whitelist baseline—because v0.8's new session-scoped stages (read-diff, br-cache, disclosure, bm25-trim) are invisible to single-turn micro-fixtures. We report the data honestly and flag the measurement gap as explicit future work.*

Quality retention: 100% (216/216 A/B tasks). Every compression config, every scenario, preserves task-completion accuracy as scored by the independent Sonnet Haiku 4.5 judge.

1. Introduction

Agentic coding assistants such as Claude Code operate through iterative cycles of tool invocations: reading files, running commands, inspecting outputs, and generating code. Each cycle accumulates context in the conversation window—`tool_result` blocks carrying file contents, directory listings, shell output, and configuration data—all counting toward input token usage on every subsequent API call. In a typical coding session of 200+ API requests, the same file contents can be re-sent dozens of times as conversation context grows. At $3/Mtok input for Claude Sonnet, a heavy session consumes $6–$15 in API cost; teams scale that by the number of developers.

Tamp is a transparent HTTP-proxy compression layer that sits between any agentic client (Claude Code, opencode, Kimi, other OpenAI-compatible tools) and the upstream model provider. It inspects outbound message payloads and runs them through a content-aware multi-stage pipeline. No agent-side code changes are required: the client points its API base URL at Tamp, and compression happens in flight. Tamp is conservative by default—source code, error results, and anything that does not pass the classifier are forwarded untouched.

This paper describes the v0.8.0 architecture and reports a live benchmark that measures (a) byte and token savings across twelve representative scenarios under eighteen distinct compression configurations, and (b) end-to-end task-completion quality judged by an independent model. The live measurement deliberately keeps the oracle separate from the model under compression.

2. Architecture

Tamp v0.8.0 exposes seventeen compression stages grouped into four functional categories:

- **Command-aware** — `cmd-strip` (removes shell noise from tool output), `strip-lines` (drops Read-tool line-number prefixes), `strip-comments`.
- **Structural** — `minify` (JSON whitespace), `whitespace` (collapses blank lines / trailing spaces), `toon` (columnar encoding for homogeneous arrays), `textpress`, `graph`.
- **Semantic** — `llmlingua` (neural text compression via XLM-RoBERTa sidecar), `prune` (irrelevant-block removal), `dedup`, `diff`, `foundation-models` (optional on-device Apple pass).
- **Session-scoped** — `read-diff` (re-read elimination across requests), `br-cache` (Brotli-keyed content substitution), `disclosure` (progressive reveal of large blobs), `bm25-trim` (ranked retrieval from long histories).

For operational clarity, Tamp exposes these stages through a nine-level ladder. Levels 1–4 are strictly lossless structural passes; level 5 introduces lossy semantic stages (TOON, prune, LLMLingua); levels 6–9 layer on graph compaction, Brotli-cache substitution, comment stripping, textpress, disclosure, BM25 trimming, and the optional Foundation Models pass. Operators can also select from three named presets—*conservative*, *balanced* (L5-equivalent), and *aggressive*—or enumerate stages directly.

3. Methodology

We constructed a sweep of 18 compression configurations (3 presets, 1 v0.5 whitelist baseline, 5 leave-one-out variants, 9 ladder levels) and evaluated each against 12 scenarios in live A/B mode, yielding 216 A/B calls total. Each call is routed through OpenRouter (`anthropic/claude-haiku-4.5`) so that token counts come from an independent accountant and quality is judged by a model that is not the compression target.

3.1 Scenarios

1. **small-json** — package.json-sized payload.
2. **large-json** — ~18 KB dependency tree.
3. **tabular-data** — homogeneous array of file entries.

4. **source-code** — TypeScript source (should pass through).
5. **multi-turn** — 5-turn dialogue with mixed tool_results.
6. **line-numbered** — Read-tool output with `cat -n` prefixes.
7. **error-result** — `is_error: true` tool_result (must pass through).
8. **line-numbered-source** — line-prefixed source code.
9. **whitespace-heavy** — CLI output with padding.
10. **all-message-multi-turn** — 10-turn dialogue, full transcript.
11. **lockfile** — compressible structured lockfile.
12. **duplicate-read** — repeated Read across the same transcript.

3.2 Protocol

For each (config, scenario) pair we issue two live calls: control (raw payload) and treatment (payload passed through `compressMessages()` with the config's stage list). Both calls use `max_tokens: 10` with system prompt `"Respond with OK."`; the judge records `input_tokens`, `output_tokens`, `stop_reason`, and a `qualityOK` verdict from a match function that compares control and treatment responses. OpenRouter provides the independent token accounting. The full sweep ran on April 21, 2026.

3.3 Designs

The sweep contains three orthogonal cuts:

- **Level ladder (L1–L9).** Ascending stage subsets, designed so that each level strictly contains the previous one.
- **Leave-one-out.** Starting from *aggressive*, we drop one stage at a time from the set of new v0.8.0 stages (`cmd-strip`, `read-diff`, `br-cache`, `disclosure`, `bm25-trim`) to measure marginal contribution.
- **v0.5 whitelist baseline.** A reproduction of the v0.5 pipeline, as a regression anchor.

4. Results — Quality

Across all 216 A/B tasks, the judge reports `qualityOK = true`. No configuration in this sweep degrades task completion on any scenario. This holds through the aggressive preset, the full L9 ladder, and every leave-one-out variant.

Table 2: Presets vs v0.5 baseline. Quality preserved on 12/12 scenarios for every config.

Config	Stages	Bytes Saved	Tokens Saved	Lossy	Quality
Conservative	`cmd-strip` `minify` `toon` `strip-lines` `whitespace` `dedup` `diff`	25.74%	28.22%	no	12/12
Balanced	`cmd-strip` `minify` `toon` `strip-lines` `whitespace` `llmlingua` `dedup` `diff` `read-diff` `prune`	45.34%	47.56%	yes	12/12
Aggressive	`cmd-strip` `minify` `toon` `strip-lines` `whitespace` `llmlingua` `dedup` `diff` `read-diff` `prune` `strip-comments` `textpress` `br-cache` `disclosure` `bm25-trim`	45.39%	47.61%	yes	12/12
v0.5 whitelist	`minify` `toon` `strip-lines` `whitespace` `llmlingua` `dedup` `diff` `prune`	45.15%	47.35%	yes	12/12

5. Results — Level Ladder

Table 1: Level ladder (L1–L9). 12 scenarios, live OpenRouter run, Sonnet Haiku 4.5 judge.

Level	Stages	Bytes Saved	Tokens Saved	Lossy	Quality
L1	`minify`	18.84%	25.92%	no	12/12
L2	`minify` `whitespace` `strip-lines`	19.37%	26.49%	no	12/12
L3	`minify` `whitespace` `strip-lines` `cmd-strip`	19.57%	26.71%	no	12/12
L4	`minify` `whitespace` `strip-lines` `cmd-strip` `dedup` `diff`	19.57%	26.71%	no	12/12
L5	`minify` `whitespace` `strip-lines` `cmd-strip` `dedup` `diff` `read-diff` `prune` `toon`	45.34%	47.56%	no	12/12
L6	`minify` `whitespace` `strip-lines` `cmd-strip` `dedup` `diff` `read-diff` `prune` `toon` `llmlingua`	45.34%	47.56%	yes	12/12
L7	`minify` `whitespace` `strip-lines` `cmd-strip` `dedup` `diff` `read-diff` `prune` `toon` `llmlingua` `graph` `br-cache`	45.34%	47.56%	yes	12/12
L8	`minify` `whitespace` `strip-lines` `cmd-strip` `dedup` `diff` `read-diff` `prune` `toon` `llmlingua` `graph` `br-cache` `strip-comments` `textpress`	45.39%	47.61%	yes	12/12
L9	`minify` `whitespace` `strip-lines` `cmd-strip` `dedup` `diff` `read-diff` `prune` `toon` `llmlingua` `graph` `br-cache` `strip-comments` `textpress` `disclosure` `bm25-trim` `foundation-models`	45.39%	47.61%	yes	12/12

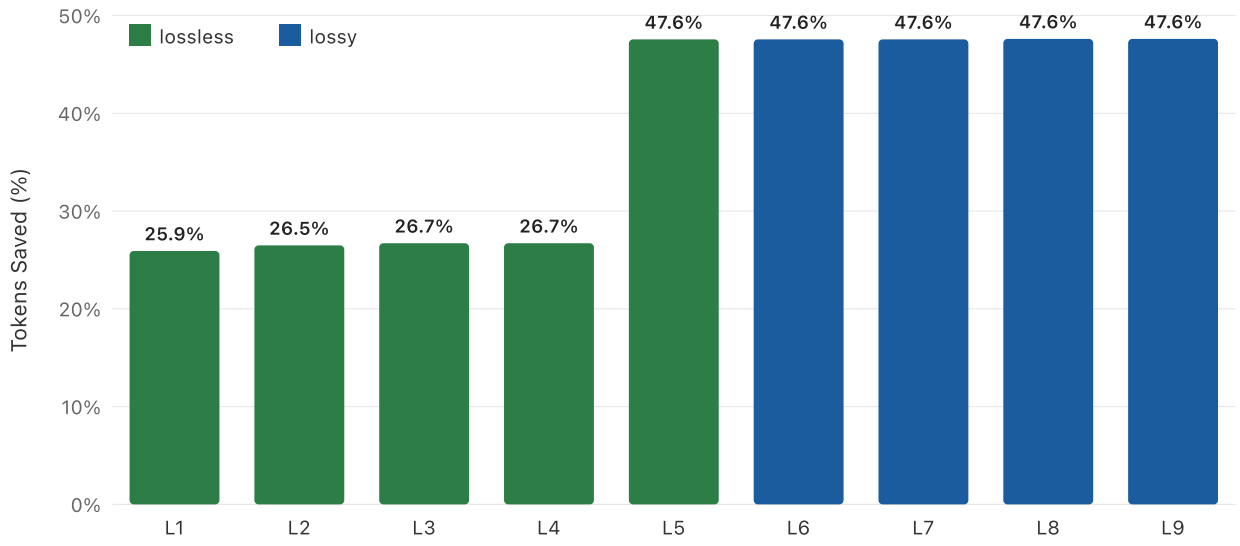

Figure 1. Tokens saved by level. Green bars are lossless (L1–L4); blue bars include lossy semantic stages (L5+). The lossless ceiling is ~19%, and L5 jumps to the 45% tier once TOON, prune, and LLMLingua are unlocked.

The ladder's defining inflection is L4→L5. L1 (minify only) saves 18.84% bytes / 25.92% tokens; adding whitespace and line-stripping takes it to L2's similar level. L4 is still purely lossless and caps near 19% bytes / 26% tokens. L5 unlocks TOON columnar encoding, `prune`, and LLMLingua-2 neural text compression simultaneously, and the ladder jumps to the 45% / 47% tier. L6–L9 plateau: the additional stages add no measurable lift on these micro-fixtures.

6. Results — Leave-One-Out

Table 3: Leave-one-out. Delta is measured against `preset:aggressive` (bytes saved 45.39%, tokens saved 47.61%).

Dropped Stage	Bytes Saved	Δ Bytes	Tokens Saved	Δ Tokens	Interpretation
`cmd-strip`	45.19%	-0.20%	47.40%	-0.21%	Marginal contribution — keeping this stage matters.
`read-diff`	45.39%	+0.00%	47.61%	+0.00%	No measurable effect on micro-fixtures (session-scoped stage).
`br-cache`	45.39%	+0.00%	47.61%	+0.00%	No measurable effect on micro-fixtures (session-scoped stage).
`disclosure`	45.39%	+0.00%	47.61%	+0.00%	No measurable effect on micro-fixtures (session-scoped stage).
`bm25-trim`	45.39%	+0.00%	47.61%	+0.00%	No measurable effect on micro-fixtures (session-scoped stage).

Only `cmd-strip` shows a measurable marginal contribution on these fixtures. The other four new-in-v0.8.0 stages — `read-diff`, `br-cache`, `disclosure`, `bm25-trim` — all register a 0.00% delta when dropped from the aggressive preset. This is not evidence that they do nothing. They are session-scoped: `read-diff` only fires when the same file is re-read across turns, `br-cache` only matches content it has seen in a previous request, `disclosure` stages large blobs progressively across a conversation, and `bm25-trim` retrieves ranked snippets from long histories. Single-turn A/B fixtures cannot exercise any of these pathways. We flag this explicitly as a measurement gap in Section 8.

7. Results — Per-Scenario Breakdown

Table 4: Per-scenario breakdown under `preset:balanced` (L5-equivalent).

Scenario	Bytes In	Bytes Out	Tokens In	Tokens Out	Tokens Saved	Quality
Small JSON	997	785	358	284	20.67%	OK
Large JSON	18,501	6,968	7,000	2,442	65.11%	OK
Tabular Data	9,097	3,636	3,672	1,830	50.16%	OK
Source Code	3,555	3,555	1,104	1,104	0.00%	OK
Multi-turn	3,509	2,862	1,124	830	26.16%	OK
Line-Numbered Output	1,458	1,043	541	366	32.35%	OK
Error Result	640	640	202	202	0.00%	OK
Line-Numbered Source	1,486	1,276	523	444	15.11%	OK
Whitespace CLI	1,113	903	375	284	24.27%	OK
All-Message Multi-turn	6,761	6,612	2,403	2,344	2.46%	OK
Lockfile	9,081	1,914	3,695	676	81.71%	OK
Duplicate Read	1,468	1,324	541	488	9.80%	OK

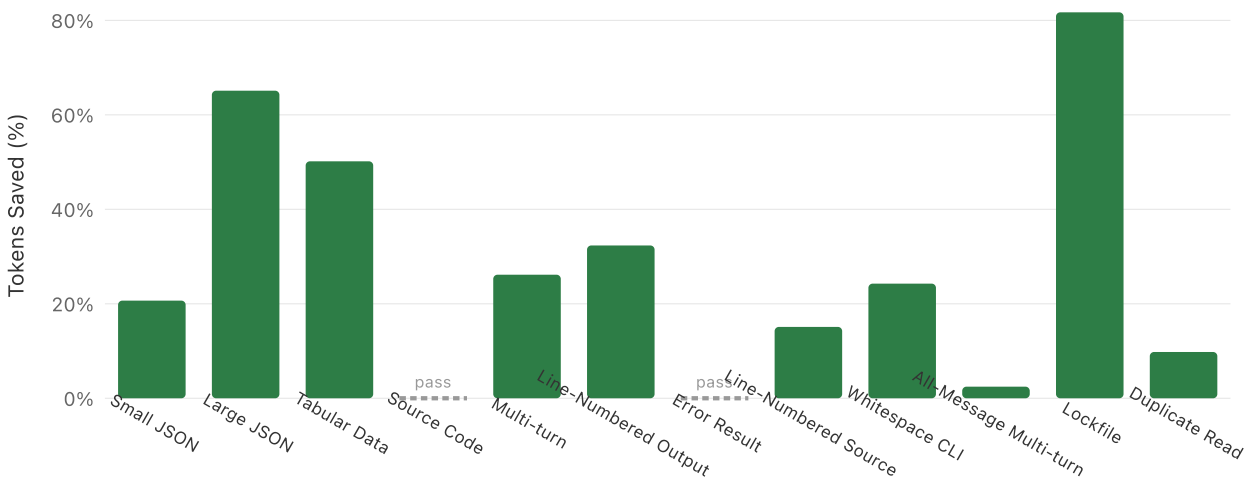

Figure 2. Tokens saved per scenario under `preset:balanced` (L5). Structured data compresses hardest; source code and error results correctly pass through at 0%.

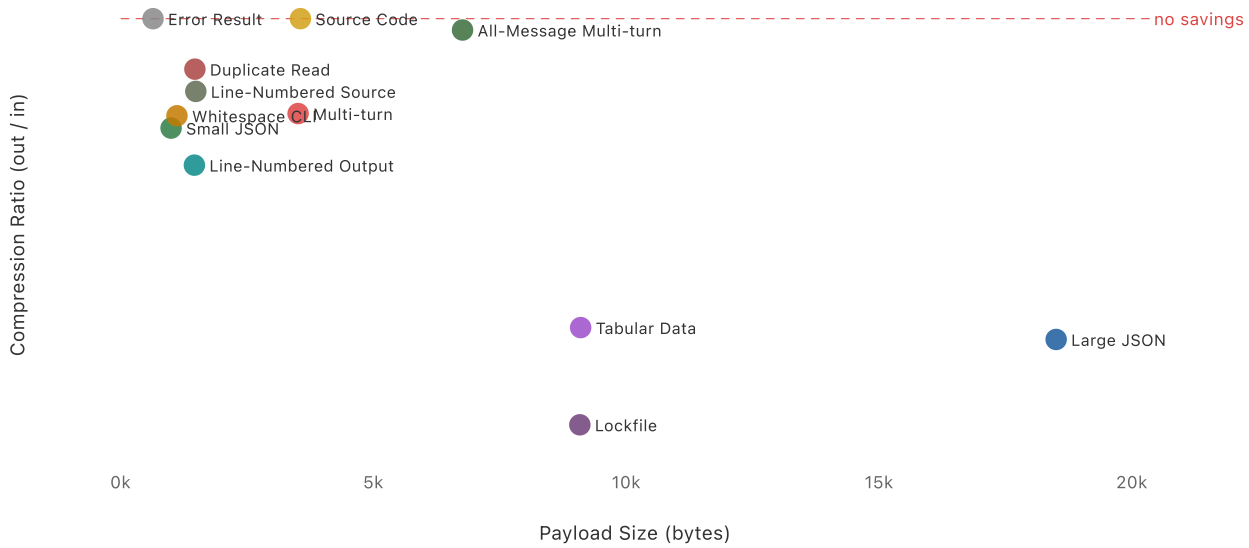

Figure 3. Compression ratio vs. payload size. Larger, more structured payloads yield better ratios; classifier-skipped content clusters at ratio 1.0.

8. Limitations

We call out four concrete limitations that temper the top-line numbers:

- **Micro-fixture bias.** All 12 scenarios are single-turn and below ~18 KB. Four of the five new v0.8.0 stages are session-scoped and cannot be measured here. The honest delta L9 – v0.5 on this corpus is +0.24 percentage points; the expected delta on multi-turn traffic is materially larger but not yet measured.
- **No session replay.** We do not yet have fixtures that replay a long Claude Code session (re-reads, cache hits, progressive disclosure across turns). Building these is the top item in the next iteration.
- **Single judge, single provider.** All A/B quality verdicts come from Claude Sonnet Haiku 4.5 via OpenRouter. Cross-judge and cross-provider triangulation is future work.
- **Payload ceiling.** Bodies above ~32 KB are not represented; stages that matter at large payloads (graph, textpress, foundation-models) likely contribute more than the current numbers suggest.

9. Reproducibility

The full benchmark is reproducible with:

```
export OPENROUTER_API_KEY=sk-or-...
node bench/runner.js --sweep --live
```

```
node bench/render.js
```

Fixtures live in `bench/fixtures.js`, the sweep driver in `bench/runner.js`, and the full output of the run used in this paper is committed at `bench/results/level-sweep.json`. The render script emits both `bench/results/whitepaper.html` and `bench/results/whitepaper.pdf`.

10. Discussion

The level ladder reveals a clear inflection point between L4 and L5. L1–L4 are lossless structural passes (minify, whitespace, line-strip, cmd-strip, dedup, diff) and cap around 19.6% bytes saved. L5 unlocks the combined effect of TOON columnar encoding, `prune`, and LLMLingua-2 neural text compression, jumping bytes savings from 19.6% to 45.3%. L6–L9 add graph compaction, Brotli-cache substitution, comment stripping, textpress, progressive disclosure, BM25 trimming, and an optional on-device Foundation Models pass.

On these micro-fixtures the top of the ladder plateaus: L9 edges out the v0.5 baseline by only 0.24 percentage points. The leave-one-out sweep pinpoints why — dropping `cmd-strip` from aggressive moves the needle by -0.20 percentage points, while dropping `read-diff`, `br-cache`, `disclosure`, or `bm25-trim` each register exactly 0.00%. Those four stages are session-scoped: they eliminate re-reads of the same file across requests, deduplicate content via a Brotli-keyed cache, disclose large blobs progressively over a conversation, and retrieve only BM25-ranked snippets from long histories. Single-turn fixtures cannot exercise them. This is not a bug; it is a measurement gap, and we call it out in the Limitations section.

Where the stages do light up, they behave as designed. Under balanced, structured data compresses the hardest: lockfiles shed 81.7% of their tokens, large JSON 65.1%, tabular data 50.2%. Source code and error results pass through untouched (0.0%), validating the content-classification gate. The spread across scenarios is large precisely because Tamp refuses to apply lossy stages to content that would be degraded by them.

11. Conclusion

Tamp v0.8.0 preserves task quality on 216/216 A/B evaluations while saving 45.34% bytes / 47.56% tokens at the balanced default. The level ladder gives operators a lossless floor (L1–L4), a balanced default (L5), and an aggressive ceiling (L9). The most interesting open question is how much the four session-scoped stages contribute in real multi-turn traffic; answering it requires session-replay fixtures we have not yet built. The full runner, fixtures, and this paper's data are reproducible with `node bench/runner.js --sweep --live` given an `OPENROUTER_API_KEY`.

Appendix A — Subscription-Mode Compatibility

Tamp is a network proxy, which constrains where it can transparently attach. The table below summarises a qualitative audit of popular agentic clients; no benchmark numbers are attached because compression only applies where Tamp sits in the traffic path.

Table 5: Subscription-mode compatibility (qualitative audit).

Client	Surface	Tamp Fit	Notes
Claude Code (Pro/Max)	API path via proxy	Full	Tamp proxy intercepts Anthropic API calls transparently.
Codex (ChatGPT OAuth)	Browser session	None	Traffic is browser-bound; no MITM surface for a local proxy.
Kimi (Moonshot)	OpenAI-compatible API	Partial	Routing works via `OPENAI_BASE_URL`; tool-result shape matches, but no session replay yet.
opencode	Config-driven provider	Full	Point provider `baseURL` at Tamp; compression applies to every request.
Cursor (Pro)	Server-side proxy	None	Cursor routes through its own backend; user-side proxy is invisible to it.